SOKOINE UNIVERSITY

SUA

OF AGRICULTURE

ABE 715: Computer Programming

Compendium Part 7

Laboratory Manual and Practical Exercises

Prepared for:
MSc. Irrigation Engineering and Management
MSc. Agricultural and Biosystems Engineering

Instructor: Dr. Kadeghe G. Fue
Electronics and Precision Agriculture Lab

Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)

Academic Year 2025/2026

Contents

Preface iii

1 Lab 1: The Engineering Environment 1
- 1.1 Objective . . . 1
- 1.2 Prerequisites . . . 1
- 1.3 Procedure . . . 1
  - 1.3.1 Step 1: Install Anaconda . . . 1
  - 1.3.2 Step 2: Create an Environment . . . 1
  - 1.3.3 Step 3: Install Libraries . . . 2
  - 1.3.4 Step 4: Verify Installation . . . 2
- 1.4 Lab Deliverable . . . 2

2 Lab 2: Meteorological Data Analysis 3
- 2.1 Objective . . . 3
- 2.2 Data Source . . . 3
- 2.3 Procedure . . . 3
  - 2.3.1 Step 1: Data Cleaning . . . 3
  - 2.3.2 Step 2: Calculate ETo . . . 4
  - 2.3.3 Step 3: Visualization . . . 4
- 2.4 Discussion Questions . . . 4

3 Lab 3: IoT Hardware Interface 5
- 3.1 Objective . . . 5
- 3.2 Equipment . . . 5
- 3.3 Wiring Diagram . . . 5
- 3.4 Procedure . . . 6
  - 3.4.1 Step 1: Arduino IDE Setup . . . 6
  - 3.4.2 Step 2: The Firmware Code . . . 6
  - 3.4.3 Step 3: Verification . . . 7

4 Lab 4: Geospatial Mapping 8
- 4.1 Objective . . . 8
- 4.2 Data . . . 8
- 4.3 Procedure . . . 8
  - 4.3.1 Step 1: Vector Analysis . . . 8
  - 4.3.2 Step 2: Plotting . . . 8
- 4.4 Lab Deliverable . . . 9

5 Lab 5: Computer Vision Disease Detection **10**
5.1 Objective 10
5.2 Procedure 10
5.2.1 Step 1: Image Acquisition 10
5.2.2 Step 2: Segmentation Script 10
5.2.3 Step 3: Calculating Health 11
5.3 Discussion 11

Preface to Part 7

This Laboratory Manual supplements the theoretical content of ABE 715. It is designed to bridge the gap between code on a screen and physical engineering reality.

The exercises contained herein require specific hardware (ESP32 microcontrollers, sensors) and software configurations (Anaconda, VS Code, QGIS/GeoPandas). Each lab is structured to be completed in a 3-hour session, with pre-lab preparation expected.

Safety Guidelines:

- When working with IoT hardware, ensure USB connections are dry and clean.
- Do not connect 5V sensors to 3.3V logic pins on the ESP32 without a level shifter.
- Handle agricultural data sets ethically; anonymize any personal farm data.

Chapter 1

Lab 1: The Engineering Environment

1.1 Objective

To set up a professional Python development environment capable of handling data science, simulation, and geospatial tasks. By the end of this lab, you will have a working installation of Anaconda and VS Code and will run your first engineering script.

1.2 Prerequisites

- A laptop (Windows, Mac, or Linux).
- Internet connection for downloading packages.

1.3 Procedure

1.3.1 Step 1: Install Anaconda

Anaconda is a package manager that simplifies installing complex libraries like Pandas and NumPy.

1. Download the installer from the official Anaconda website.
2. Run the installer. Keep default settings.
3. Open "Anaconda Navigator" to verify installation.

1.3.2 Step 2: Create an Environment

We create a virtual environment to keep our project dependencies isolated. Open **Anaconda Prompt** (Windows) or Terminal (Mac/Linux) and run:

```
conda create --name abe715 python=3.9
conda activate abe715
```

1.3.3 Step 3: Install Libraries

Install the core engineering stack:

```
conda install numpy pandas matplotlib scipy seaborn
conda install -c conda-forge geopandas rasterio
pip install paho-mqtt opencv-python
```

1.3.4 Step 4: Verify Installation

Create a test script `check_env.py` in VS Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("Environment Setup Successful!")
print(f"NumPy Version: {np.__version__}")

# Simple Plot Test
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y)
plt.title("Test Plot")
plt.show()
```

1.4 Lab Deliverable

Submit a screenshot of the sine wave plot generated by your local machine.

Chapter 2

Lab 2: Meteorological Data Analysis

2.1 Objective

To use Pandas and Matplotlib to clean, analyze, and visualize real-world weather station data. You will calculate Reference Evapotranspiration (ETo) trends.

2.2 Data Source

Use the file `morogoro_weather_2024.csv` provided on the course portal. Columns: `Date`, `T_max`, `T_min`, `RH_mean`, `Wind_Speed`, `Solar_Rad`.

2.3 Procedure

2.3.1 Step 1: Data Cleaning

Load the data and handle missing values.

```
import pandas as pd

# Load
df = pd.read_csv("morogoro_weather_2024.csv")

# Convert Date
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

# Check for missing values
print(df.isnull().sum())

# Fill missing temperature with the previous day's value (
    Forward Fill)
df.fillna(method='ffill', inplace=True)
```

2.3.2 Step 2: Calculate ETo

Implement a simplified Penman-Monteith function (refer to Compendium Part 6, Appendix B).

```
def simple_eto(row):
    # Simplified coefficients for lab demonstration
    t_mean = (row['T_max'] + row['T_min']) / 2
    radiation = row['Solar_Rad']
    # formula approximation
    eto = 0.0023 * (t_mean + 17.8) * (radiation ** 0.5) * \
          (row['T_max'] - row['T_min'])**0.5
    return eto

# Apply to every row
df['ETo'] = df.apply(simple_eto, axis=1)
```

2.3.3 Step 3: Visualization

Create a dual-axis plot showing Temperature and ETo.

```
import matplotlib.pyplot as plt

fig, ax1 = plt.subplots(figsize=(10,6))

color = 'tab:red'
ax1.set_xlabel('Date')
ax1.set_ylabel('Max Temp (C)', color=color)
ax1.plot(df.index, df['T_max'], color=color)
ax1.tick_params(axis='y', labelcolor=color)

ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('ETo (mm/day)', color=color)
ax2.plot(df.index, df['ETo'], color=color, linestyle='--')
ax2.tick_params(axis='y', labelcolor=color)

plt.title("Morogoro Climate Analysis 2024")
plt.tight_layout()
plt.show()
```

2.4 Discussion Questions

1. Why did we use 'Forward Fill' for missing temperature data instead of filling with Zero?

2. Observe the graph. How does ETo correlate with Maximum Temperature?

Chapter 3

Lab 3: IoT Hardware Interface

3.1 Objective

To program an ESP32 microcontroller to read a DHT22 sensor and transmit the data via MQTT. This simulates a wireless field node.

3.2 Equipment

- ESP32 Development Board
- DHT22 (Temperature/Humidity Sensor)
- Jumper Wires & Breadboard
- USB Cable

3.3 Wiring Diagram

Use the following pin mapping:

DHT22 Pin	ESP32 Pin
VCC	3.3V
GND	GND
DATA	GPIO 4

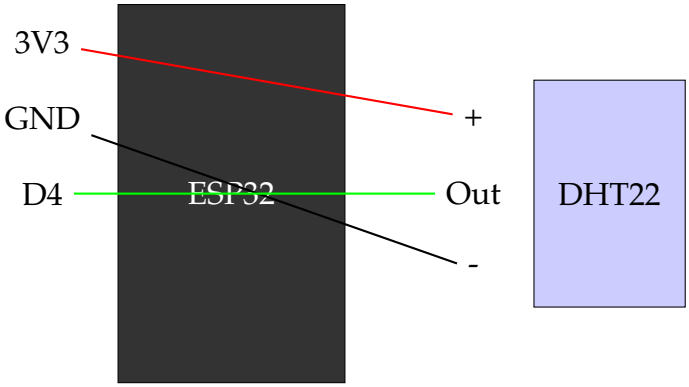


Figure 3.1: Wiring Diagram for ESP32 and DHT22

3.4 Procedure

3.4.1 Step 1: Arduino IDE Setup

1. Install the "ESP32 by Espressif Systems" board manager.
2. Install the "DHT sensor library" by Adafruit.
3. Install the "PubSubClient" library (for MQTT).

3.4.2 Step 2: The Firmware Code

Flash this code to the ESP32.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT22

const char* ssid = "SUA_Student_WiFi";
const char* password = "password123";
const char* mqtt_server = "broker.hivemq.com";

DHT dht(DHTPIN, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.println("Connecting to WiFi...");
  }
```

```
25
26    client.setServer(mqtt_server, 1883);
27  }
28
29  void loop() {
30    if (!client.connected()) {
31      // Reconnect logic here
32      if (client.connect("ESP32_SUA_Student")) {
33          Serial.println("MQTT Connected");
34      }
35    }
36
37    float h = dht.readHumidity();
38    float t = dht.readTemperature();
39
40    String payload = String("{\"temp\":") + t + ",\"hum\":" + h
        + "}";
41    client.publish("sua/lab/dht", payload.c_str());
42
43    delay(2000);
44  }
```

3.4.3 Step 3: Verification

Open the Serial Monitor (Baud Rate 115200). You should see "Connecting..." followed by successful sensor readings. Use a Python script (from Lecture 14) on your laptop to verify you are receiving the data.

Chapter 4

Lab 4: Geospatial Mapping

4.1 Objective

To use GeoPandas to visualize agricultural plots and calculate vegetation indices.

4.2 Data

- `farm_boundary.geojson`: A vector file of the university farm.
- `sentinel_image.tif`: A raster image of the farm area.

4.3 Procedure

4.3.1 Step 1: Vector Analysis

Load the farm boundary and calculate the acreage.

```
import geopandas as gpd

farm = gpd.read_file("farm_boundary.geojson")

# Reproject to UTM Zone 37S (Meters) for accurate area
farm = farm.to_crs(epsg=32737)

area_sqm = farm.geometry.area.sum()
area_acres = area_sqm / 4046.86

print(f"Total Farm Area: {area_acres:.2f} Acres")
```

4.3.2 Step 2: Plotting

Visualize the farm.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 10))
```

```
farm.plot(ax=ax, color='white', edgecolor='black')
plt.title("SUA Model Farm Boundaries")
plt.xlabel("Easting")
plt.ylabel("Northing")
plt.grid(True)
plt.show()
```

4.4 Lab Deliverable

Submit a PDF report containing:

1. The code used for area calculation.
2. The map generated in Step 2.
3. A brief explanation of why EPSG:32737 (UTM) was used instead of EPSG:4326 (Lat/Lon).

Chapter 5

Lab 5: Computer Vision Disease Detection

5.1 Objective

To build a simple "Greenness" detector that can identify healthy vs. stressed plants using OpenCV.

5.2 Procedure

5.2.1 Step 1: Image Acquisition

Take a photo of a plant leaf against a white background using your phone. Save it as `leaf.jpg` in your project folder.

5.2.2 Step 2: Segmentation Script

Write a script to separate the green leaf from the background.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('leaf.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Define Green Color Range
lower_green = np.array([35, 40, 40])
upper_green = np.array([85, 255, 255])

# Mask
mask = cv2.inRange(hsv, lower_green, upper_green)
result = cv2.bitwise_and(img, img, mask=mask)

# Display Side by Side
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original")

plt.subplot(1, 2, 2)
plt.imshow(mask, cmap='gray')
plt.title("Green Mask")

plt.show()
```

5.2.3 Step 3: Calculating Health

Count the pixels.

```
# Total pixels in image
total_pixels = img.shape[0] * img.shape[1]

# Green pixels (Non-zero in mask)
green_pixels = cv2.countNonZero(mask)

ratio = green_pixels / total_pixels
print(f"Green Cover Ratio: {ratio:.4f}")
```

5.3 Discussion

If the leaf has yellow spots (disease), they will not be included in the mask. Therefore, a lower "Green Cover Ratio" relative to the leaf's total size indicates disease. How would you modify the code to specifically detect Yellow spots? (Hint: Change the HSV bounds).